

INSTITUTE OF ENGINEERING & MANAGEMENT

Department of Computer Science and Engineering
(Artificial Intelligence & Machine Learning)
IEM Newtown Campus

ASSIGNMENT 2

Lab Report: Synchronization & Explicit Waits in Selenium

Name:	Roshni Ghosh Tagore
Enrollment No:	12023002028175
Course:	Python for Automation
Academic Year:	2023–2027
Department:	Computer Science and Engineering (Artificial Intelligence & Machine Learning)
Campus:	IEM Newtown

Module Name: Tier 1: Core Fundamentals & Locators

Experiment Title: Synchronization & Explicit Waits (Assignment 2)

1. Problem Statement

Modern web applications heavily rely on asynchronous JavaScript (AJAX) and dynamic DOM updates, causing elements to render after variable time delays. The task requires automating interaction with a dynamically loading page:

- Navigate to a dynamic content page (e.g., `https://the-internet.herokuapp.com/dynamic_loading/1`).
- Trigger the dynamic event by clicking the "Start" button.
- **Constraint:** Strictly avoid using `time.sleep()`.
- Implement `WebDriverWait` in combination with `expected_conditions` (EC) to wait until the dynamically revealed text element becomes visible.
- Extract and print the revealed text content once available in the DOM.

2. Objective

- To master synchronization strategies in automated UI testing using Selenium.
- To understand the drawbacks of hardcoded pauses (`time.sleep()`) and implement non-blocking polling mechanisms.
- To utilize `WebDriverWait` and `expected_conditions.visibility_of_element_located` for efficient element synchronization.
- To reliably retrieve dynamically loaded text content from asynchronous web pages.

3. Tools, Software, and Concepts Used

- **Programming Language:** Python
- **Automation Library:** Selenium WebDriver (`selenium.webdriver`)
- **Synchronization Modules:** `WebDriverWait` and `expected_conditions` (EC)
- **IDE / Editor:** Visual Studio Code (VS Code)
- **Target Web Application:** Herokuapp Dynamic Loading Page (`https://the-internet.herokuapp.com/dynamic_loading/1`)
- **Key Concepts:** Explicit Waits, Dynamic DOM Synchronization, Conditional Polling, Element Visibility Conditions, Resource Cleanups.

4. Source Code / Implementation Steps

```
from selenium import webdriver
from selenium.webdriver.common.by import By
from selenium.webdriver.support.ui import WebDriverWait
from selenium.webdriver.support import expected_conditions as EC

driver = webdriver.Chrome()

try:
    # Navigate to dynamic loading page
    driver.get("https://the-internet.herokuapp.com/dynamic_loading/1")

    # Click start button
    driver.find_element(By.CSS_SELECTOR, "#start button").click()

    # Wait until text element is visible
    wait = WebDriverWait(driver, 10)
    text_element = wait.until(
        EC.visibility_of_element_located((By.XPATH, "//div[@id='finish']/h4"))
    )

    # Extract text
    extracted_text = text_element.text
    print(extracted_text)

finally:
    driver.quit()
```

5. Output with Clearly Labelled Screenshots

```
assignment1.py > ...
1  from selenium import webdriver
2  from selenium.webdriver.common.by import By
3
4  driver = webdriver.Chrome()
5
6  try:
7      # 1. Navigate to target URL
8      driver.get("https://www.saucedemo.com/")
9      driver.maximize_window()
10
11     # 2. Interact using By.ID, By.NAME, and By.XPATH
12     driver.find_element(By.ID, "user-name").send_keys("standard_user")
13     driver.find_element(By.NAME, "password").send_keys("secret_sauce")
14     driver.find_element(By.XPATH, "//input[@id='login-button']").click()
15
16     # 3. Validation
17     current_url = driver.current_url
18     assert "/inventory.html" in current_url, f"Expected '/inventory.html' in URL, got {current_url}"
19     print("Assignment 1 Passed: Login successful and URL validated.")
20
21 finally:
22     driver.quit()
```

Screenshot 1: Python Source Code Implementation in Visual Studio Code (assignment2.py)

```
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> python -u "c:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots\assignment1.py"
Assignment 1 Passed: Login successful and URL validated.
(.venv) PS C:\Users\roshn\OneDrive\Documents\Desktop\ALL SELENIUM\ScreenShots> 
```

Screenshot 2: Terminal Execution Output verifying successful explicit wait and text extraction ("Hello World!")

6. Brief Result, Observation, and Conclusion

Result:

The script executed cleanly without throwbacks or time-out exceptions. Upon clicking the "Start" button, the script dynamically waited for the AJAX loading bar to finish and extracted the revealed text, printing: **"Hello World!"** in the terminal console.

Observation:

- `WebDriverWait` polls the DOM at default intervals (500ms) until the expected condition evaluates to true or the 10-second timeout expires.
- Using `EC.visibility_of_element_located` guarantees that the element is not only present in the DOM but also visible to the user before interaction.
- Avoiding `time.sleep()` optimizes test suite execution time by continuing immediately once the element becomes visible rather than waiting for a fixed duration.

Conclusion:

The experiment successfully validated the implementation of Explicit Waits in Selenium WebDriver. Explicit synchronization ensures robust, flake-free automation scripts capable of seamlessly handling dynamic and asynchronous content loading.